# TileGrad 1.0 Language Specification

A tile language over tinygrad UOps

TileGrad contributors

July 2026

## Scope

TileGrad is a Python-embedded language for high-performance tiled kernels. Programs describe logical tiles, memory placement, launch geometry, and scheduling intent. The TileGrad compiler validates each program, assigns work to lanes, inserts required synchronization, legalizes layouts, and selects instructions. It then lowers the program to tinygrad UOps. tinygrad renders, compiles, and executes those UOps on the selected device.

This document specifies the planned TileGrad 1.0 language, not the current experimental implementation. The emphasized keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY** indicate normative requirements. Exact Python signatures and target records are defined in the API and capability references.

A versioned numerical manifest defines the numerical requirements, and conformance tests verify them. Each conformance revision pins the manifest and the supported tinygrad revisions.

Portable operations have target-independent semantics and provide a scalar or SIMT lowering. An implementation **MAY** choose a different legal lowering for performance, but that choice **MUST NOT** produce observable results outside the numerical contract. Expert operations expose target-specific behavior only through versioned capabilities. Legalization **MUST** reject an expert operation when its requested capability is unavailable.

## Programs

A decorated function accepts runtime tensors and scalars and **MAY** also accept compile-time parameters. It declares its outputs, contains one or more kernel launch regions, and returns those outputs.

Calling the function specializes the program, compiles and caches the specialization, and executes it. Kernel regions are submitted in source order. Each region depends on every preceding region whose effects it consumes. When a dependent-launch capability is available, an implementation **MAY** overlap regions only when doing so preserves those dependencies.

A returned tensor retains the device dependencies of every region that produced it. Returning the tensor does not require host synchronization. Workgroups within a launch are unordered unless an advertised cooperative capability provides an ordering mechanism.

| Value | Meaning | Legal use |
|---|---|---|
| Compile-time | Constant included explicitly in the specialization key. | Python control, shapes, layouts, workgroup shape, unrolling, and scheduling. |
| Shape-specialized | Bound from tensor shape or stride and included in the key. | The same uses as compile-time values after binding. |
| Runtime | Device scalar not specialized by value. | Device expressions, predicates, indices, and dynamic serial bounds. It cannot determine storage or launch shapes. |

An external tensor has a dtype, rank, shape, strides, device, and access mode. Portable conformance requires dense, contiguous external tensors. Strided external tensors require an advertised capability. An input is read-only unless declared writable.

The host wrapper allocates each output unless the caller supplies it. Outputs **MUST NOT** alias any input or other output unless the declarations permit the alias. The function returns outputs in source order. A rank-zero tensor represents a scalar device result.

Repeated symbols in declarations **MUST** bind to the same value. The specialization key includes every source value and environment field that can affect legalization or code generation. These fields include dtype, rank, device, numerical mode, capability registry, and the TileGrad and TINYGRAD revisions.

```
@tilegrad.jit
def matmul(A, B, *, BM: T.Const[int]=64,
           BN: T.Const[int]=64, BK: T.Const[int]=32):
  M, N, K = T.const("M", "N", "K")
  A: T.Tensor[(M, K), T.float16]
  B: T.Tensor[(K, N), T.float16]
  C = T.empty((M, N), T.float16)

  with T.Kernel(T.ceildiv(N, BN), T.ceildiv(M, BM),
                threads=128) as (bx, by):
    As = T.shared((BM, BK), T.float16)
    Bs = T.shared((BK, BN), T.float16)
    acc = T.fragment((BM, BN), T.float32)
    T.clear(acc)
    for ko in T.Pipelined(T.ceildiv(K, BK), stages=3):
      T.copy(A.tile((by*BM, ko*BK), (BM, BK)), As, pad=0)
      T.copy(B.tile((ko*BK, bx*BN), (BK, BN)), Bs, pad=0)
      T.mma(As, Bs, acc)
    T.copy(acc, C.tile((by*BM, bx*BN), (BM, BN)))
  return C
```

## Values and Storage

A view is the tuple $(B, o, \mathbf{s}, \mathbf{d})$, where $B$ is the base storage, $o$ is an element offset, $\mathbf{s}$ is the logical shape, and $\mathbf{d}$ contains the element strides. A view also has a dtype, rank, address space, access mode, layout, bounds, and effects.

A coordinate $\mathbf{i}$ is in the logical domain when $0 \le i_k < s_k$ for every axis. Its mapped address is $o + \sum_k i_k d_k$. A coordinate in the logical domain is storage-valid only when its mapped address lies within the base allocation. An operation **MAY** use a logical coordinate that is not storage-valid only when that operation defines masking, padding, or access-suppression semantics.

| Space | Ownership | Purpose |
|---|---|---|
| Global | Program or invocation | Inputs, outputs, and workspaces. |
| Shared | Workgroup or cluster | Cooperative staging and communication. |
| Fragment | Participant group | Lane-distributed values and accumulators. |
| Private | One local agent | Scalars and statically indexed register storage. |
| Reducer | Participant group | Reduction or scan state. |
| Barrier/descriptor | Target-defined participant group | Async transactions, matrix operations, and target storage. |
| Target | Named capability | Tensor memory or another advertised physical scope. |

Local allocations have shapes fixed during specialization and lexical lifetimes. Dynamic shared storage requires a capability. A fragment is one shaped logical value. Legalization assigns each fragment coordinate to one physical owner unless a replicated layout states otherwise. An implementation **MUST NOT** use complete per-lane replication as a fallback unless the layout requests it. Private dynamic indexing **MAY** spill only when the source or conformance profile permits spilling.

TILE, SUBVIEW, TRANSPOSE, RESHAPE, and VIEW create aliases. They do not copy data. RESHAPE requires a physical layout that the compiler can prove compatible. VIEW **MAY** reinterpret the dtype only when the total byte size is unchanged and the required alignment is satisfied.

A tile **MAY** include coordinates that are not storage-valid, but each operation that consumes those coordinates **MUST** mask, pad, or suppress the corresponding accesses. Between synchronization points, each non-atomic writable address **MUST** have exactly one dynamic writer. Views with zero strides, negative strides, or overlapping coordinates are otherwise read-only.

A copy between overlapping source and destination regions requires snapshot mode. In snapshot mode, every logical source value is read before any destination value is written. If the compiler cannot prove this ordering safe, it **MUST** reject the copy.

## Layouts

A layout maps each logical coordinate to one or more participant-slot pairs. Multiple pairs are permitted only for a replicated layout. Layouts apply to parallel loops, fragments, shared storage, and matrix atoms. When the source does not specify a layout, the compiler **MUST** infer a legal one. The resolved mapping **MUST** be inspectable.

The source **MAY** request a canonical row-major or column-major distribution, a composition of reshape, repeat, and permutation operations, a shared-memory swizzle, or a named capability layout.

Layouts **MUST** cover each logical coordinate exactly as required by the operation. A layout **MUST NOT** introduce data races or make observable results target-dependent. Code that combines raw thread access with logical parallel access **MUST** name a compatible layout. The language supports arbitrary target-independent layout composition. A layout tied to an instruction, warp size, bank geometry, or descriptor format requires a named capability.

## Control and Scheduling

| Form | Semantics |
|---|---|
| Serial | Ordered iteration over a half-open integer interval. |
| Unroll | Serial semantics with a request to expand iterations during specialization. |
| Vectorized | Serial element semantics with a request to use packed instructions. |
| Parallel | Unordered logical Cartesian iteration distributed by a layout. |
| Reduce | A recurrence that **MAY** be reassociated under the numerical contract. |
| Pipelined | Serial semantics with an inferred pipeline depth or source-specified stages and order. |
| Persistent | Logical work items assigned statically or dynamically to resident workgroups. Under the capability's progress model, each item executes exactly once in a successful invocation. |

Device control includes conditionals and selects with runtime predicates, while loops with runtime conditions, and break and continue statements in serial and while loops. Divergent control flow is legal unless it contains a collective whose convergence the compiler cannot prove. The compiler folds compile-time conditions during specialization. Host Python helpers run only during specialization. Device macros are typed, hygienic, and inlined during specialization.

Pipeline scheduling applies to effectful statements. The compiler versions affected storage and emits the required prologue, steady state, and epilogue. When the selected capability provides asynchronous operations, the compiler **MAY** insert the corresponding commits, waits, and barriers. Otherwise, it executes the pipeline synchronously while preserving serial semantics. Pure scalar expressions **MAY** be recomputed at their points of use.

A source-specified stage or order **MUST** respect every data and effect dependency. Coalescing, cache policy, rasterization, occupancy, warp specialization, and register allocation are non-semantic scheduling hints unless the source requests an exact capability. Autotuning **MAY** search compile-time parameters and scheduling hints. The chosen configuration and resulting legalized IR **MUST** be reproducible and inspectable.

Raw local IDs are available to manual workgroup code. Access to lane, subgroup, warp, warp-group, cluster, and grid IDs requires expert capabilities with declared geometry. These IDs do not replace logical PARALLEL. Code that uses them is responsible for mapping assumptions not represented by a layout.

## Tile Operations

An agent is one local execution instance. A participant group is the fixed set of agents that take part in one dynamic collective instance. The collective is convergent when every member executes that instance exactly once with compatible parameters.

Each operation declares whether it is per-agent or collective. Scalar expressions, pointwise fragment maps, scalar memory operations, and atomics **MAY** appear in divergent control flow. Cooperative copies and transforms, MMA operations, distributed reductions and scans, and synchronization operations are collective. Their default participant group is the workgroup. A target **MAY** use smaller native groups only when legalization partitions the logical operation without exposing that choice to the program.

| Operation | Inputs | Semantics |
|---|---|---|
| Fill/Clear | destination, scalar | Write every storage-valid destination coordinate; clear fills with zero. |
| Copy | source, destination, pad, hints | Synchronously transfer equal-shaped regions. Use the pad value for source coordinates that are not storage-valid and suppress writes to destination coordinates that are not storage-valid. |
| AsyncCopy | source, destination, hints | Start an advertised transfer and return a token. An access that requires transfer completion is invalid until a wait on the token provides the required completion guarantee. |
| Transpose/Transform | source, destination, map | Cooperative data rearrangement, including transpose and im2col-style maps. |
| MMA | A, B, accumulator, transforms | Accumulate logical matrix multiplication; no implicit clear. |
| SparseMMA | values, metadata, B, accumulator | Accumulate a declared structured sparse matrix multiplication. |
| Map/Select/Cast | fragments or scalars | Pointwise operation with right-aligned broadcasting. The operation returns a new logical value. |
| Reduce | source, axes, op, dtype | Sum, product, min, max, or bitwise reduction with a specified accumulator dtype and optional retained axes. |
| Scan | source, axis, op, direction | Inclusive or exclusive prefix operation; segmented scans are a capability. |
| Load | view, coordinates, mask, alternate | Return the alternate value when the mask is false or the mapped address is invalid. A masked-off load need not form an address. |
| Store | view, coordinates, value, mask | Write only when the mask is true and the mapped address is valid. |
| Atomic | operation-specific operands, order, scope | Atomic load/store, RMW, and compare-exchange. The operation defines whether it returns the old value or a success result. |
| Diagnose | condition, values/message | Bounded device assertion and debug output. The implementation **MAY** use a debug buffer when target diagnostics are unavailable. |

The destination of Copy is ready for use when the operation returns, even if its lowering uses asynchronous instructions.

AsyncCopy exposes asynchronous execution to expert code. Legalization **MUST** reject an AsyncCopy whose scope, shape, alignment, or target is not supported. It **MUST NOT** replace the operation with a synchronous copy. A copy **MAY** infer its extent when one shaped operand determines that extent. Broadcasting requires a broadcast view.

Transfer hints **MAY** specify a layout, vector width, cache policy, or preferred mechanism. A hint does not affect program semantics unless it requests an exact mechanism. A transaction capability **MAY** define descriptors, expected-byte-count or arrival-count accounting, commit groups, parity-based or count-based waits, and separate completion points for source reads and destination writes. Each token identifies its transaction family and the completion guarantee provided by waiting on that token.

MMA supports operand transposition, a specified accumulator dtype, and an optional instruction mode. Portable dense MMA **MUST** have a SIMT lowering. SparseMMA requires a capability that defines its metadata format and the semantics of any permitted fallback.

Elementwise operations include arithmetic, comparisons, logical operations, clamp, select, casts, exponentials, logarithms, trigonometry, reciprocal, square root, and common activation functions. Dtype-specific dot products,

packed quantization, random generation, and custom transforms **MUST** be expressed as compositions or capabilities rather than untyped source injection.

## Effects and Synchronization

An effect names a base storage region, access kind, participant group, and memory scope. Within one agent, or within an ordered collective operation, source order defines the sequenced-before relation. A release operation synchronizes with a matching acquire operation at a compatible scope. Happens-before is the transitive closure of sequenced-before and synchronizes-with.

Two unordered conflicting accesses form a data race when at least one is a non-atomic write. A program with a data race is invalid. Separate workgroups **MUST** communicate only through atomics or an advertised cooperative or cluster mechanism.

The supported atomic orders are relaxed, acquire, release, acquire-release, and sequentially consistent. Legalization **MUST** reject any unsupported combination of operation, order, and scope.

Managed storage uses compiler-planned synchronization. The compiler **MUST** preserve read-after-write and write-after-write dependencies, loop-carried dependencies, and write-after-read dependencies before reusing storage. The source **MAY** select manual synchronization for an allocation or region. Manual synchronization **MUST NOT** be mixed with managed access unless an explicit operation transfers ownership.

Each barrier and asynchronous token declares its participant group, memory scope, and ordering semantics. A plain TINYGRAD BARRIER provides only the workgroup semantics guaranteed by the pinned adapter. Other scopes require typed UOps admitted by the integration contract.

When advertised by the target, the language supports subgroup, workgroup, cluster, and cooperative grid barriers. It also supports arrival and wait barriers for asynchronous transactions. Every participant **MUST** reach each barrier convergently.

Votes, ballots, shuffles, lane matching, and predicated block synchronization are participant-group capabilities. Each such capability defines its width and mask. None assumes a universal warp size.

## Capabilities

A capability is a versioned, namespaced contract among TILEGRAD, a target, and a compatible TINYGRAD revision. It identifies the target predicate and operation. It also defines the supported dtypes, shapes or atoms, participant geometry, operand scopes, layouts, alignment, effects, required UOps, and numerical behavior.

The mode `auto` permits any compatible capability or portable lowering. The mode `portable` requests scalar or SIMT execution. An exact capability request **MUST NOT** fall back to another lowering. If the requested capability is unavailable, legalization **MUST** report the source operation, expected contract, observed types and shapes, target, and missing feature.

The legalized IR **MUST** show the selected capability and every layout, barrier, and storage allocation inserted by the compiler.

Standard capability families are:

- native matrix operations represented through TINYGRAD WMMA, including MMA/WMMA, WGMMA, MFMA, TCGEN05, and simdgroup families;
- cooperative/vector memory operations, asynchronous copy, TMA-like multidimensional transfer, proxy fences, shared swizzles, and cache controls;
- structured sparse matrix operations and low-precision packed formats;
- subgroup collectives, atomics, memory barriers, clusters, cooperative grids, dependent launch, warp specialization, and persistent scheduling; and
- descriptor-managed or target storage such as tensor memory, including allocation, lifetime, fences, and register-budget controls.

These families describe feature classes, not NVIDIA source APIs. Support for a family is optional unless a conformance profile requires it. A backend **MUST** expose only families admitted by the pinned integration contract. Each exposed family requires typed TINYGRAD UOps and renderer, compiler, and runtime support.

Capability adapters define target-specific descriptor fields, fence sequences, and instruction schemas. Backend source injection is not a TileGrad language capability.

## Numerics

Scalar types are bool, signed and unsigned integers, float16, bfloat16, float32, and float64. Float8, sub-byte integer and floating-point types, packed vectors, and mixed formats require a capability. A Python literal converts to the type required by its context; every other type change uses a cast or bitcast. Index values are integral.

Each reduction and MMA specifies its associative operator, identity, empty-domain result, and accumulator dtype. Integer addition, subtraction, multiplication, negation, and bitwise operations are modular. Separate rules define the results or failure conditions for division, shifts, conversion, and index overflow. Index arithmetic **MUST NOT** wrap to produce a valid address. Floating-point operations are defined by their real-valued formulas.

The numerical mode selects strict or fast behavior. The lowering mode independently selects portable, auto, or an exact capability. A capability is eligible only when it satisfies the selected numerical mode.

The normative numerical manifest defines rounding, reassociation, contraction, approximation error, denormal handling, infinity, NaN, signed zero, and per-operation tolerances.

## Lowering

| Stage | Responsibility |
|---|---|
| Specialize | Bind compile-time and shape-specialized values and form the cache key. |
| Typed tile IR | Verify types, shapes, views, effects, uniformity, and resources. |
| Scheduled tile IR | Resolve logical layouts, pipelines, storage versions, and managed synchronization. |
| Legalized tile IR | Select a portable lowering or an exact capability and make participant ownership, masks, barriers, and target storage explicit. |
| TINYGRAD UOps | Emit only operations admitted by the pinned integration contract, including optional typed UOps for atomic, synchronization, transaction, and WMMA operations. |
| Program | Delegate optimization, rendering, compilation, launch, and execution to a compatible TINYGRAD device/runtime. |

Every named IR stage **MUST** be inspectable and verifiable. The compiler **MUST** eliminate all tile-only nodes before the UOp boundary. A conforming TILEGRAD/TINYGRAD pairing **MUST** preserve emitted memory effects and synchronization and reject any requested mechanism that it cannot represent.

The implementation **MUST** distinguish among invalid source programs, unsupported target capabilities, invalid UOp construction, compilation failures, and runtime failures.

## Conformance

An implementation claiming **portable conformance MUST** support copies, views, layouts, fragment algebra, reductions, scans, dense GEMM, and numerically stable causal and non-causal FlashAttention using scalar or SIMT paths. Tests cover masks, tails, invalid aliases, data races, and divergent collectives.

An implementation claiming **accelerated conformance MUST** demonstrate a native matrix capability implemented with WMMA UOps, cooperative vector copy, distributed native fragments, and inspectable multi-stage storage with ordered commit and wait operations. The same inputs, shapes, and masks **MUST** be exercised through both paths under the same numerical manifest entry.

**Expert conformance** is claimed separately for each advertised capability family. Each claim **MUST** include successful execution, rejection of unsupported shapes, types, and targets, effect and synchronization tests, inspectable legalization, and a pinned TINYGRAD revision range. Asynchronous transfer, structured sparse MMA, atomics, subgroup exchange, clusters, cooperative launch, and descriptor-managed storage are independent claims.

The portable workload gates are edge-tiled mixed-precision GEMM; reductions and scans with irregular tails; and numerically stable FlashAttention with masking, online softmax, two dense MMAs, and a log-sum-exp result computed using the natural logarithm. Structured sparse GEMM gates the sparse-MMA expert claim.

Implicit autograd is not required. Explicit backward kernels are ordinary TILEGRAD programs. The conformance suites, API reference, capability registry, and integration contract are versioned with this document.